PORTFOLIO PROJECT · BUSINESS ANALYSIS + ENGINEERING

Filing a court case without going to court

A plain-language guide to how I redesigned a slow paper process and built the system behind the new one.

The problem, through one person's eyes

Sana's tenant hasn't paid rent in eight months. To start a case today, she has to:

1. Prepare a paper file
2. Travel to court and queue at the counter
3. Sent home if one page is missing
4. Pay the fee at a bank, come back with the receipt
5. Visit again just to ask what happened

2+
trips to court before her case even has a number

0
updates unless she asks in person

1 pile
urgent cases (evictions, demolitions) wait with everything else

The goal I set: Sana gets a case number, her fee and progress updates without visiting the court. Registrars receive cases already sorted, with urgent ones on top.

THE IDEA

Think of a busy restaurant

A good restaurant doesn't make you stand at the counter while your meal is cooked. The waiter checks your order, gives you a number straight away, and pins the ticket on the rail. Each kitchen station picks it up, does its part and passes it on. You sit down and get called when it's ready.

The new filing service works the same way:

In the restaurant	In the court service
Waiter checks the order is complete	The online form checks every required document is there, instantly
You get a ticket number	Sana gets her case number in a fraction of a second
Ticket rail in the kitchen	A **message queue**: a to-do list that each team picks work from
Kitchen stations (grill, salad, dessert)	Automatic helpers: one sorts the case, one works out the fee, one sends it to the right registrar
"Order 42 is ready!"	Sana gets an email at each step
The order book	A **database** that keeps each case as one complete folder

Why the "ticket rail" matters

Sana never waits for the slow parts

She only waits for the check and the number. Sorting, pricing and routing happen in the background.

Nothing gets lost

If a helper is down, the ticket stays on the rail. It is retried, and if it still fails, set aside for a person to look at.

Busy days are fine

Add more helpers and they share the rail. No one has to redesign anything.

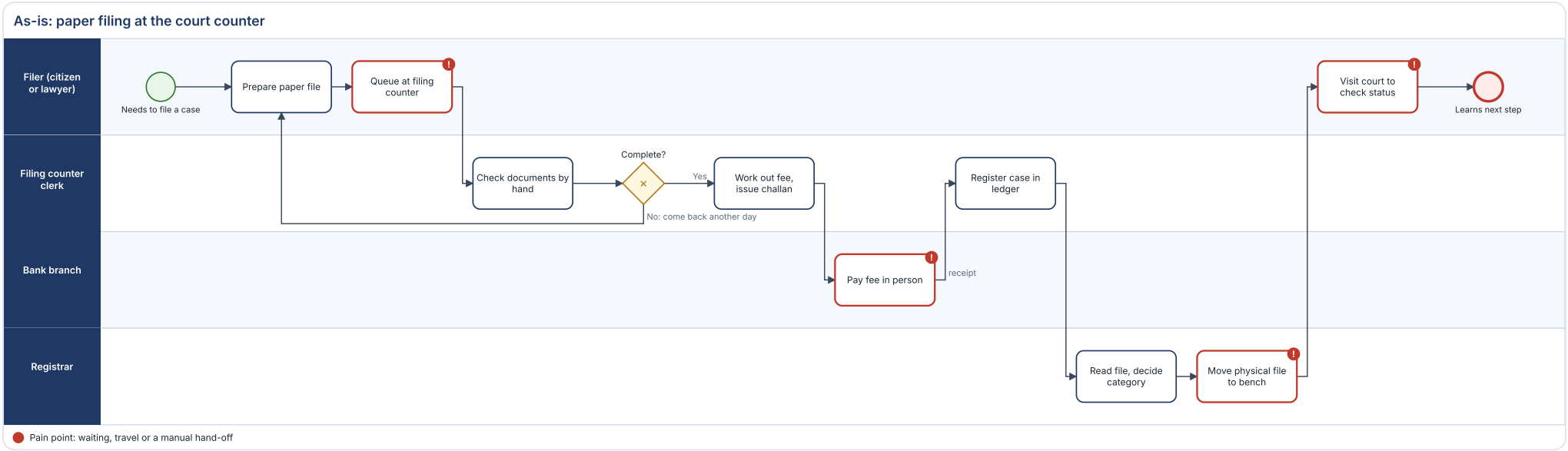
Every step is on record

Each change is written to a log that can't be edited, which a court needs for audits.

BEFORE AND AFTER

Before: paper filing

I drew both processes as **swimlane maps**: each horizontal lane is one person or system, so you can see every hand-off. Red marks are where people wait, travel or carry paper.



Four of the eleven steps are waiting, travel or a manual hand-off. The filer appears at the start and the end, with no updates in between.

Checks happen too late

A missing page is found only at the counter, after the trip, so the filer goes home and starts again.

Money moves on foot

The fee is worked out by hand, paid at a bank and proven with a paper receipt.

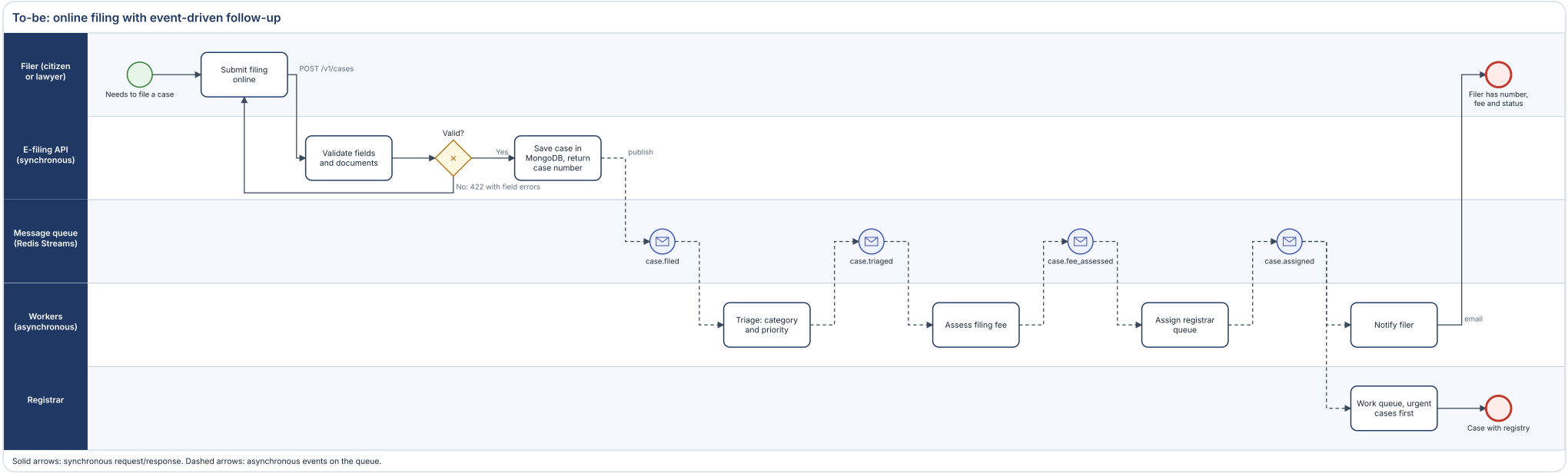
No one tells the filer anything

The only way to learn the next step is another visit or a phone call.

BEFORE AND AFTER

After: online filing

Sana only waits for the top two lanes. Everything below the line happens in the background, one ticket at a time.



Solid arrows: Sana waits for these (a fraction of a second). Dashed arrows: tickets on the rail, handled in the background. No one carries a file anywhere.

Checked at the door

Missing documents are flagged on screen before submitting, so no wasted trips.

Sorted and priced automatically

Each case gets a category, an urgency level and a fee the moment it is filed.

Updates come to you

An email when the case is received, when the fee is set and when it reaches the registry.

MY APPROACH

From a messy process to working software

1. **Understand the people.**
   Listed everyone affected (citizens, lawyers, clerks, registrars, finance, IT) and what each one needs.

2. **Map today's process.**
   Drew the as-is swimlane and marked every point where someone waits, travels or re-does work.

3. **Design the new process.**
   Split the work in two: what the filer must wait for (checks, case number) and what can happen in the background (sorting, fee, routing, updates).

4. **Write it down so it can be built.**
   12 requirements, 6 user stories with "given, when, then" tests, rules for speed, reliability and privacy, and the exact format each part of the system sends and receives.

5. **Plan for things going wrong.**
   Double-clicked submit, a helper crashing, the queue going down, old records in an older format. Each one has a rule and a safety net.

6. **Build it.**
   A working service with a database for cases and a message queue between the helpers.

7. **Prove every requirement.**
   Each requirement has its own automatic test, and a checklist links requirement, story, process step, code and test. If a requirement has no test, the build fails.

What happened when I ran it with five sample cases

The case, in short	Sorted as	Priority	Fee	Sent to
Tenant won't pay rent or leave	Property	Standard	6,300	Civil registry A
Buyer's cheque bounced on a contract	Commercial	Standard	39,500	Commercial registry
Salary and gratuity withheld	Labour	Standard	0 (free)	Labour registry
Stop demolition of a family home	Property	Urgent	1,500	Civil registry A
Child custody and maintenance	Family	Standard	500	Family registry

Each filer got a case number in a few milliseconds and three email updates. Fees and categories are illustrative; the scenario is fictional.

UNDER THE HOOD, BRIEFLY

The safety nets

Pressed "submit" twice?

Each submission carries a one-time key. The second press returns the same case number instead of creating a duplicate.

A helper keeps failing?

The ticket is retried three times, then moved to a "needs attention" pile. The case stays exactly where it stopped, so nothing is half-done.

The ticket rail is down?

The case is still saved and Sana still gets her number. A clean-up job re-sends the missing tickets later.

Old records in an old format?

When the data format changed, older cases are converted as they're read, so nothing needs a risky mass update.

What I would do next

Smarter sorting

Train a model on past cases to sort new ones, with a clerk deciding whenever the model isn't confident.

Pay online

A payment helper that listens for "fee set" and posts "fee paid", so cases reach the registrar only once paid.

Measure the before

Count today's trips, days to registration and rejections before launch, so the improvement is proven.

Skills this project shows

Process mapping (swimlanes, BPMN) · As-is / to-be analysis · Requirements and user stories · Acceptance criteria · Non-functional requirements · Requirements traceability · API design and contracts · Data modelling and versioning · NoSQL (MongoDB) · Message queues (Redis Streams) · Event-driven design · Python, FastAPI · Automated testing

See it all: github.com/ayeshafaheem72/court-efiling-event-pipeline
The README links to the process maps, requirements, data model, API contract and tests.